

[BE-3] Session Based Authentication & Authorization Fiber

1. Persiapan

Untuk mengikuti pembelajaran ini kamu perlu menyiapkan:

- Bahasa **Go Lang** (<https://go.dev/>).
- **Git** untuk clone starter proyek (<https://git-scm.com/>).
- **VS Code** untuk text editor, atau editor kesukaanmu (<https://code.visualstudio.com/>). Pastikan menginstall Plugin Go Lang.
- **Postman** untuk test API yang kita buat nanti (<https://www.postman.com/>).

2. Autentikasi dan Autorisasi untuk Web API

Autentikasi dan autorisasi adalah dua konsep penting dalam pengembangan Web API yang aman. **Autentikasi** adalah proses verifikasi identitas pengguna, memastikan bahwa pengguna adalah benar-benar siapa yang mereka klaim, biasanya dilakukan melalui kredensial seperti username dan password. Sementara itu, **otorisasi** adalah proses penentuan hak akses setelah pengguna terautentikasi, mengatur apa yang boleh dan tidak boleh dilakukan oleh pengguna tersebut dalam sistem. Kedua mekanisme ini bekerja bersama untuk menciptakan lapisan keamanan yang komprehensif dalam Web API, di mana autentikasi mengidentifikasi pengguna dan otorisasi mengontrol akses mereka ke resource dan endpoint tertentu.

a. Session Based Authentication

Merupakan metode autentikasi tradisional dimana server menyimpan data sesi pengguna setelah login berhasil. Server akan membuat session ID unik dan mengirimkannya ke client dalam bentuk cookie. Setiap request berikutnya, client akan mengirimkan cookie yang berisi session ID ini untuk memverifikasi identitasnya.

b. Token Based Authentication

Berbeda dengan session-based, token-based authentication tidak menyimpan status sesi di server. Setelah autentikasi berhasil, server membuat token (biasanya JWT/Paseto) yang berisi informasi pengguna dan mengirimkannya ke client. Token ini kemudian dikirim dalam header Authorization pada setiap request berikutnya untuk memverifikasi identitas pengguna.

Metode Autentikasi	Kelebihan	Kekurangan
Session Based	• Mudah diimplementasikan • Kontrol penuh dari sisi server • Mudah mencabut akses (invalidate session) • Penggunaan memori efisien untuk data sederhana	• Membutuhkan penyimpanan di server • Sulit untuk scaling horizontal • Tidak cocok untuk aplikasi mobile • Rentan terhadap CSRF attacks
Token Based	• Stateless dan mudah untuk scaling • Cocok untuk arsitektur microservices • Bekerja baik dengan mobile apps • Cross-domain authentication	• Token size bisa besar • Tidak bisa mencabut token sebelum expired • Kompleksitas implementasi lebih tinggi • Perlu manajemen token yang baik

Menariknya, Fiber sendiri sudah mempunyai library built-in untuk session based authentication yang bernama **Fiber Session** (<https://docs.gofiber.io/api/middleware/session/>).

3. Tabel User

- a. Buat model untuk user, buat file `user.go` di folder model

```
type User struct {
    Id          uuid.UUID `gorm:"type:uuid;default:uuid_generate_v4();primaryKey" json:"id"`
    Username    string    `json:"username"`
    Name        string    `json:"name"`
    Password    string    `json:"- "`
```

```

    CreatedAt time.Time `gorm:"default:now();" json:"created_at"`
    UpdatedAt *time.Time `json:"updated_at"`
}

func (u *User) HashPassword() error {
    hashedPw, err := bcrypt.GenerateFromPassword([]byte(u.Password), 12)
    if err != nil {
        return err
    }
    u.Password = string(hashedPw)
    return nil
}

func (u *User) CheckPassword(password string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(u.Password), []byte(password))
    return err == nil
}

```

Jangan lupa tambahkan ke auto migrate di function `InitDB()`

```

func InitDB() {
    ...
    database.AutoMigrate(
        &model.Book{},
        &model.User{},
    )
    ...
}

```

b. Jalankan proyek untuk melihat perubahan

4. Session Based Auth di Framework Fiber

a. Buat file `session.go` di package DB untuk initiate store yang akan menyimpan session di sisi backend.

```

package db

import "github.com/gofiber/fiber/v2/middleware/session"

var Store *session.Store

func InitStore() {
    session := session.New(session.Config{
        CookieHTTPOnly: true,
        CookieSecure:    true,
    })

    Store = session
}

```

b. Panggil fungsi `InitStore()` di `main.go`

```

// Inisialisasi Store
db.InitStore()

```

c. Tambahkan session ke dalam fiber context untuk diakses nantinya, tambahkan code berikut di `main.go`

```

// Tambahkan session ke fiber context
app.Use(func(c *fiber.Ctx) error {
    sess, err := db.Store.Get(c)

```

```

        if err != nil {
            return err
        }
        c.Locals("session", sess)
        return c.Next()
    })

```

- d. Buat utility untuk membuat dan mendapatkan sesi, buat file `session.util.go` dalam folder `utils`

```

func GenerateSession(c *fiber.Ctx, user model.User) error {
    sess := c.Locals("session").(*session.Session)

    sess.Set("username", user.Username)
    sess.Set("user_id", user.Id.String())

    // Save session
    if err := sess.Save(); err != nil {
        return fmt.Errorf("failed to create session")
    }

    return nil
}

func ValidateSession(c *fiber.Ctx) bool {
    sess := c.Locals("session").(*session.Session)

    username := sess.Get("username")
    id := sess.Get("user_id")
    return !(username == nil || id == nil)
}

```

5. Register & Login API

- a. Buat repository dan implementation untuk user

- `user.repository.go`

```

type UserRepository interface {
    GetUserByUsername(username string) (*model.User, error)
    AddUser(user *model.User) error
}

```

- `user.repository.impl.go`

```

type userRepositoryImpl struct{}

// AddUser implements UserRepository.
func (u userRepositoryImpl) AddUser(user *model.User) error {
    err := db.DB.Create(user).Error
    return err
}

// GetUserByID implements UserRepository.
func (u userRepositoryImpl) GetUserByUsername(username string) (*model.User, error) {
    user := new(model.User)
    if err := db.DB.First(user, "username = ?", username).Error; err != nil {
        return nil, err
    }
    return user, nil
}

```

```

}

func NewUserRepository() UserRepository {
    return userRepositoryImpl{}
}

```

b. Buat Controller untuk Register & Login, kita beri nama `auth.controller.go`

```

// Field Untuk request
type (
    RegisterReq struct {
        Username string `json:"username"`
        Name      string `json:"name"`
        Password  string `json:"password"`
    }

    LoginReq struct {
        Username string `json:"username"`
        Password string `json:"password"`
    }
)

type authController struct {
    userRepository repository.UserRepository
}

func NewAuthController(repo repository.UserRepository) authController {
    return authController{
        userRepository: repo,
    }
}

func (ac authController) Register(c *fiber.Ctx) error {
    userReq := new(RegisterReq)

    if err := c.BodyParser(userReq); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Code:    fiber.StatusBadRequest,
            Message: "Mohon isikan data dengan benar",
        })
    }

    valid := validateUser(userReq)
    if !valid {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Message: "Mohon isikan data dengan benar",
        })
    }

    user := model.User{
        Username: userReq.Username,
        Name:     userReq.Name,
        Password: userReq.Password,
    }

    if err := user.HashPassword(); err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Error{
            Message: "Terjadi Kesalahan",
        })
    }
}

```

```

    }

    if err := ac.userRepository.AddUser(&user); err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Error{
            Message: "Terjadi Kesalahan",
        })
    }

    return c.JSON(fiber.Map{
        "message": "Pendaftaran sukses",
    })
}

func (ac authController) Login(c *fiber.Ctx) error {
    userReq := new(LoginReq)

    if err := c.BodyParser(userReq); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Code:    fiber.StatusBadRequest,
            Message: "Mohon isikan data dengan benar",
        })
    }

    user, err := ac.userRepository.GetUserByUsername(userReq.Username)
    if err != nil {
        return c.Status(fiber.StatusNotFound).JSON(fiber.Error{
            Message: "Pengguna tidak terdaftar!",
        })
    }

    valid := user.CheckPassword(userReq.Password)
    if !valid {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Message: "Password salah!",
        })
    }

    if err := utils.GenerateSession(c, *user); err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Error{
            Message: err.Error(),
        })
    }

    return c.JSON(fiber.Map{
        "message": "Login sukses",
    })
}

func validateUser(user *RegisterReq) bool {
    return !(user.Username == "" || user.Name == "" || user.Password == "")
}

```

c. Tambahkan Route untuk Register & Login

```

func SetupAuthRoute(r *fiber.App) {

    userRepository := repository.NewUserRepository()
    authController := controller.NewAuthController(userRepository)

```

```

    auth := r.Group("/auth")
    auth.Post("/register", authController.Register)
    auth.Post("/login", authController.Login)
}

```

d. Jangan lupa tambahkan route ke `main.go`

```

routes.SetupAuthRoute(app)

```

6. Middleware

Middleware adalah komponen penting dalam pengembangan web yang berfungsi sebagai perantara antara request dan response. Dalam konteks autentikasi, middleware dapat digunakan untuk memverifikasi apakah pengguna sudah login sebelum mengakses endpoint tertentu.

Mari kita buat middleware untuk mengamankan endpoint yang membutuhkan autentikasi. Middleware ini akan memastikan bahwa hanya user yang terdaftar dan telah login saja yang boleh mengakses route buku yang telah kita tambahkan sebelumnya.

1. Buat file `auth.middleware.go` di folder middleware

```

package middleware

import (
    "github.com/gofiber/fiber/v2"
    "github.com/gofiber/fiber/v2/middleware/session"
)

func AuthMiddleware() fiber.Handler {
    return func(c *fiber.Ctx) error {
        valid := utils.ValidateSession(c)
        if !valid {
            return c.Status(fiber.StatusUnauthorized).JSON(fiber.Error{
                Message: "You are not allowed to access this",
            })
        }

        return c.Next()
    }
}

```

2. Tambahkan middleware ke setiap route yang ada di `book.route.go`

```

func SetupBookRoute(r *fiber.App) {
    r.Get("/", func(c *fiber.Ctx) error {
        return c.SendString("Belajar bareng API Sederhana!")
    })

    bookRepository := repository.NewBookRepositoryPostgres()
    bookController := controller.NewBookController(bookRepository)

    book := r.Group("/book", middleware.AuthMiddleware())

    // Get Buku
    book.Get("/", bookController.GetBook)

    // Get buku berdasarkan ID
    book.Get("/:id", bookController.GetBookByID)
}

```



```
// Post Buku
book.Post("/", bookController.AddBook)

// Update buku
book.Patch("/:id", bookController.UpdateBook)

// Hapus buku
book.Delete("/:id", bookController.DeleteBook)
}
```

7. Apa Selanjutnya?

- Membuat kepemilikan buku berdasarkan user